Planteamiento y arquitectura

1. Planteamiento del problema

Una empresa tiene facturas en MongoDB. Las que están en `primerrecordatorio` o `segundorecordatorio` deben notificarse por correo al cliente, y **solo después de enviar el correo** se actualiza su estado en la base de datos. El proceso debe funcionar con varios clientes y tener una interfaz gráfica con el resumen de las facturas.

Requisito del enunciado	Cómo se resolvió
Conectarse a MongoDB y leer las facturas de un cliente	`FacturaRepository.ObtenerPorClienteAsync` (MongoDB.Driver)
Revisar que el estado sea `primerrecordatorio` o `segundorecordatorio`	`EstadoFactura.RequiereRecordatorio` (dominio)
Enviar un email según el estado encontrado	`MensajeRecordatorio.Crear` arma el correo; `SmtpEmailSender` lo envía (MailKit)
Actualizar el estado **después** de enviar	`ProcesadorRecordatorios`: primero `EnviarAsync`, luego `ActualizarEstadoAsync`
`primerrecordatorio` → `segundorecordatorio` y `segundorecordatorio` → `desactivado`	`EstadoFactura.Siguiente`
Funcionar con 3 clientes distintos	Procesa todos o uno solo; los datos demo incluyen 3 clientes con 7 facturas
Resumen de facturas en una interfaz gráfica	Frontend Angular 20, con datos de `GET /api/facturas`
Hecho en Visual Studio con .NET y C#	Solución `Recordatorios.sln` (.NET 9), probada en Visual Studio 2022

2. Flujo del proceso

```
POST /api/recordatorios/procesar
  └─ ProcesadorRecordatorios
       por cada cliente → por cada factura del cliente:
         ¿estado en recordatorio?  no → se ignora
                                   sí → 1. construye el correo según el estado actual
                                        2. envía el correo (SMTP)
                                        3. SOLO si se envió: actualiza el estado en MongoDB
                                        si algo falla → registra el error, NO cambia el estado
                                                        y sigue con la siguiente factura
```

3. Arquitectura en capas (Clean Architecture)



Capa	Contiene	Depende de
Domain	`Cliente`, `Factura`, `EstadoFactura` (regla de transición)	nada
Application	Casos de uso (`ProcesadorRecordatorios`, `ConsultaFacturas`), plantilla del correo y las **interfaces** (`IClienteRepository`, `IFacturaRepository`, `IEmailSender`, `IDatosDemo`)	Domain
Infrastructure	MongoDB.Driver (repositorios, datos demo) y MailKit (envío SMTP)	Application
Api	Controllers, configuración, inyección de dependencias y CORS	Application, Infrastructure

Regla de dependencia: las flechas apuntan siempre hacia adentro. El dominio no sabe que existen MongoDB, SMTP ni HTTP, así que se pueden cambiar sin tocar la lógica de negocio.

4. Principios SOLID aplicados

Principio	Dónde se ve
S (una responsabilidad)	`MensajeRecordatorio` solo arma correos; `ProcesadorRecordatorios` solo orquesta; los repositorios solo acceden a datos; `EstadoFactura` solo decide transiciones.
O (abierto/cerrado)	Cambiar MongoDB o el proveedor de correo = escribir otra implementación de la interfaz, sin tocar Application. *Limitación honesta:* agregar un estado nuevo exige editar `EstadoFactura` y `MensajeRecordatorio`; una mejora sería una estrategia por estado.
L (sustitución)	Cualquier `IEmailSender` o `IFacturaRepository` es intercambiable; los tests usan dobles (NSubstitute) en su lugar.
I (interfaces pequeñas)	Cuatro interfaces cortas y específicas, en vez de una "interfaz de todo".
D (inversión de dependencias)	Application define las interfaces y depende de ellas, no de MongoDB ni de MailKit; Infrastructure las implementa.

5. Inyección de dependencias

- Las dependencias se reciben por **constructor** (por ejemplo `ProcesadorRecordatorios(IClienteRepository, IFacturaRepository, IEmailSender, IOptions<ConfiguracionCorreo>, ILogger<…>)`).
- Se registran en `Program.cs` y en `Infrastructure/DependencyInjection.cs` (`AddInfrastructure`), con el contenedor nativo de .NET.

- La configuración (MongoDB, SMTP, datos de empresa) llega por el patrón *Options* (`IOptions<T>`), no por valores fijos en el código.

6. Modelo de datos (MongoDB)

Base `recordatorios`, con dos colecciones:

Colección	Campos
`clientes`	`_id`, `nombre`, `email`
`facturas`	`_id`, `clienteId`, `numero`, `monto` (Decimal128), `fechaVencimiento`, `estado`

`estado` guarda exactamente `primerrecordatorio`, `segundorecordatorio` o `desactivado`. Otros valores (por ejemplo `pagada`) se ignoran.

7. Decisiones de diseño

- **Estado después del correo:** si el estado se cambiara antes y el correo fallara, la factura quedaría marcada como notificada sin haberlo sido. Se prefiere fallar "hacia atrás".
- **Un fallo no detiene el lote:** cada factura se procesa de forma independiente y el resultado devuelve éxito o error por factura.
- **Riesgo conocido (entrega "al menos una vez"):** si el correo se envía pero la actualización en MongoDB falla, la próxima ejecución volvería a enviar ese correo. Una solución para producción sería un registro de envíos (patrón *outbox*).
- **Correos:** HTML (tabla con el detalle de la factura, color según el aviso) y también versión en texto plano. El contenido del cliente se escapa para evitar inyección de HTML.
- **Secretos fuera del repositorio:** `dotnet user-secrets` o variables de entorno. La configuración por defecto de desarrollo usa MongoDB y SMTP locales, para que el proyecto corra sin cuentas externas.
- **Datos demo:** `POST /api/demo/reiniciar` (solo en Development) recarga 3 clientes con 7 facturas.

8. Pruebas

Proyecto	Pruebas	Qué cubren
`Recordatorios.Tests` (xUnit, NSubstitute, FluentAssertions)	20	Reglas de transición de estados; el orden correo → actualización; que un fallo de correo no cambie el estado y el lote siga; cliente inexistente; contenido y escape del correo
`frontend` (Jasmine/Karma)	7	Agrupación por cliente, filtro, búsqueda, resumen, conteos y error de API

Además se verificó de punta a punta, con MongoDB Atlas y Gmail reales, con MongoDB y Mailpit locales, con los contenedores de `docker-compose.yml` y desde una copia limpia del repositorio.

9. Limitaciones y siguientes pasos

- Ejecutar el proceso de forma automática (diaria) con un `BackgroundService` o un job programado.

- Reintentos y cola de envío ante fallos temporales del SMTP.
- Pruebas de integración de los repositorios con MongoDB real (Testcontainers).
- Autenticación en la API y registro de envíos (*outbox*) para evitar duplicados.